Lenguajes no regulares

Clase 7: Lenguajes independientes de contexto

¿Cuándo un lenguaje es independiente de contexto?

Cuando son generados por una gramática donde las reglas de producción tienen la forma:

$$A \rightarrow \alpha$$

donde α es una cadena (posiblemente vacía) de variables y terminales.

Una gramática con ese tipo de reglas es una gramática independiente de contexto (GIC)

Observación: las reglas de una gramática regular también tienen esa forma, por lo tanto, toda gramática regular es también independiente de contexto.

Y todo lenguaje regular también es lenguaje independiente de contexto

Ejemplos de GIC y LIC

Sea L = $\{a^n b^n, n \geq 0\}$

Ese lenguaje se puede generar con la GIC G = {V,Σ,S,P}
V = {S}
Σ = {a,b}
P = {S $\rightarrow$ aSb
S $\rightarrow$ λ}

Claramente, L = L(G)

Construir árbol de derivación de las cadenas: λ, ab, aabb.

Ejemplos de GIC y LIC

Considere la GIC G = {V,Σ,S,P}

V = {S}

Σ = {a,b}

P = {$S \rightarrow aSa$

$S \rightarrow bSb$

$S \rightarrow \lambda$}

Genera el lenguaje de cadenas capicúas?

Ejemplos de GIC y LIC

Sea $L = \{\alpha \in \Sigma^* : \alpha = \alpha^R\}$

Ese lenguaje se puede generar con la GIC $G = \{V,\Sigma,S,P\}$

$V = \{S\}$

$\Sigma = \{a,b\}$

$P = \{S \rightarrow aSa$

$S \rightarrow bSb$

$S \rightarrow a$

$S \rightarrow b$

$S \rightarrow \lambda\}$

Claramente, L = L(G)

Construir árbol de derivación de las cadenas: aba, abaaba, abbba.

Ejemplos de GIC y LIC

Sea L = {cadenas de “if”, “else”, “end” válidas}. Reglas de las cadenas:

Balance: Todo if debe cerrarse con un único end posterior.

Rama opcional: Entre un if y su end correspondiente puede aparecer **como máximo un** else.

Anidamiento y secuencia: Dentro del cuerpo del if y dentro del cuerpo del else (así como a nivel principal) pueden ir cero, uno o más bloques if-end o if-else-end anidados/concatenados.

Ese lenguaje se puede generar con la GIC G = {V,Σ,S,P}

V = {S,A}

Σ = {if, else, end}

P = {S $\rightarrow$ S A | λ

A $\rightarrow$ if S end | if S else S end }

Ejemplos de GIC y LIC

¿Es correcta esta gramática para ese lenguaje?

- Chequear que genere sólo cadenas válidas
- Chequear que genere todas las cadenas válidas.

V = {S,A}

Σ = {if, else, end}

P = {S $\rightarrow$ if A end | S S

A $\rightarrow$ else A

A $\rightarrow$ if A end

A $\rightarrow$ λ}

GIC para lenguajes de programación

La sintaxis de los lenguajes de programación, (o de gran parte de ellos) se presenta usualmente por medio de gramáticas GIC. En tales casos se dice que el lenguaje está en la forma **Backus-Naur** o, simplemente, en la forma **BNF**.

Los lenguajes que están en BNF ofrecen ventajas significativas para el diseño de analizadores sintácticos en compiladores.

En el contexto de los lenguajes de programación los terminales de la gramática sintáctica (la que define las reglas de la sintaxis en el lenguaje) se denominan también componentes léxicos o tokens:

⟨ident⟩, ⟨sgn⟩, ⟨expresion⟩, ⟨real⟩, etc...

Ejemplo GIC números reales

V={⟨real⟩, ⟨dígitos⟩, ⟨decimal⟩, ⟨exp⟩, ⟨sgn⟩}
Σ={0,1,2,3,4,5,6,7,8,9,.,+,-,E}
S = ⟨real⟩

⟨real⟩ → ⟨sgn⟩⟨dígitos⟩⟨decimal⟩⟨exp⟩
⟨dígitos⟩ → ⟨dígitos⟩⟨dígitos⟩ |0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
⟨decimal⟩ → . ⟨dígitos⟩ |λ
⟨exp⟩ → E⟨dígitos⟩ |E+⟨dígitos⟩ |E-⟨dígitos⟩ |λ
⟨sgn⟩ → + |-|λ

Genera cadenas como 34.2, 27.0E-2, -1.54E1
No genera cadenas como .325, E125, 42.5E, 0.1E+

Ejemplo GIC expresiones aritméticas

Σ = {0,1,+,*,(,)}
V = {⟨expresión⟩, ⟨término⟩, ⟨factor⟩, ⟨número⟩}
S = ⟨expresión⟩
P:
⟨expresión⟩ → ⟨término⟩ | ⟨expresión⟩+⟨término⟩
⟨término⟩ → ⟨factor⟩ | ⟨término⟩*⟨factor⟩
⟨factor⟩ → ⟨número⟩ |(⟨expresión⟩)
⟨número⟩ → 1⟨número⟩ |0⟨número⟩ |0 | 1

Genera cadenas como 10+101*10, (101+10*10)
No genera cadenas como +10, 1*(0+)

Ejemplo de una gramática no GIC

La gramática $G = \{V,\Sigma,S,P\}$

$V = \{S,B,C\}$

$\Sigma = \{a,b,c\}$

$P = \{S \rightarrow aSBC$

$S \rightarrow abC$

$CB \rightarrow BC$

$bB \rightarrow bb$

$bC \rightarrow bc$

$cC \rightarrow cc\}$

reglas sensibles al contexto

Esta gramática genera $L(G) = \{a^k b^k c^k, k \geq 1\}$

Unión de LICs. Ejemplo

Consideremos

$L_1 = \{a^n b^m, m > n \geq 0\}$ y $L_2 = \{a^n b^m, n = m\}$

y las GIC que los generan:

$G_1 = \{V_1, \Sigma, S_1, P_1\}$

$V_1 = \{S_1, B\}$

$P_1 = \{S_1 \rightarrow aS_1b \mid B$

$B \rightarrow bB \mid b\}$

$G_2 = \{V_2, \Sigma, S_2, P_2\}$

$V_2 = \{S_2\}$

$P_2 = \{S_2 \rightarrow aS_2b \mid \lambda\}$

$L_1 \cup L_2 = \{a^n b^m, m \geq n \geq 0\}$ es generado por la gramática:

$G = \{V, \Sigma, S, P\}$

$V = \{S, S_1, B, S_2\}$

$P = \{S \rightarrow S_1 \mid S_2, \quad S_1 \rightarrow aS_1b \mid B, \quad B \rightarrow bB \mid b, \quad S_2 \rightarrow aS_2b \mid \lambda\}$

Unión de LICs es LIC

Sean L_1 y L_2 dos LIC, generados por $G_1 = \{V_1,\Sigma,S_1,P_1\}$ y $G_2 = \{V_2,\Sigma,S_2,P_2\}$ respectivamente (para evitar colisiones, V_1 y V_2 no tienen elementos el común).

Construiremos una GIC que genere $L_1 \cup L_2$

$V = V_1 \cup V_2 \cup \{S\}$ (es una nueva variable, no presente en $V_1 \cup V_2$)

$P = P_1 \cup P_2 \cup \{S \rightarrow S_1 , S \rightarrow S_2 \}$

$L(G) = L_1 \cup L_2$

Entonces, la unión de LIC es LIC.

Concatenación de LICs. Ejemplo

Consideremos

$L_1 = \{a^n b^n, n \geq 0\}$ y $L_2 = \{a^n b^{2n}, n \geq 0\}$

y las GIC que los generan:

$G_1 = \{V_1, \Sigma, S_1, P_1\}$

$V_1 = \{S_1, B\}$

$P_1 = \{S_1 \rightarrow aS_1b \mid \lambda\}$

$G_2 = \{V_2, \Sigma, S_2, P_2\}$

$V_2 = \{S_2\}$

$P_2 = \{S_2 \rightarrow aS_2bb \mid \lambda\}$

$L_1 L_2 = \{a^n b^n a^m b^{2m}, m,n \geq 0\}$ es generado por la gramática:

$G = \{V, \Sigma, S, P\}$

$V = \{S, S_1, S_2\}$

$P = \{S \rightarrow S_1S_2, \quad S_1 \rightarrow aS_1b \mid \lambda, \; S_2 \rightarrow aS_2bb \mid \lambda\}$

Concatenación de LICs es LIC

Sean L_1 y L_2 dos LIC, generados por $G_1 = \{V_1,\Sigma,S_1,P_1\}$ y $G_2 = \{V_2,\Sigma,S_2,P_2\}$ respectivamente (para evitar colisiones, V_1 y V_2 no tienen elementos el común)

Construiremos una GIC que genere $L_1\ L_2$

$V = V_1 \cup V_2 \cup \{S\}$ (es una nueva variable, no presente en $V_1 \cup V_2$)

$P = P_1 \cup P_2 \cup \{S \rightarrow S_1\ S_2\ \}$

$L(G) = L_1\ L_2$

Entonces, la concatenación de LIC es LIC.

Clausura de LICs es LIC

Sea L_1 un LIC, generados por $G_1 = \{V_1,\Sigma,S_1,P_1\}$. Construiremos una GIC que genere L_1^*.

$V = V_1 \cup \{S\}$ (S es una nueva variable, no presente en V_1)

$P = P_1 \cup \{S \rightarrow S_1\ S,\ S \rightarrow \lambda\}$

$L(G) = L_1^*$

Entonces, la clausura de LIC es LIC.

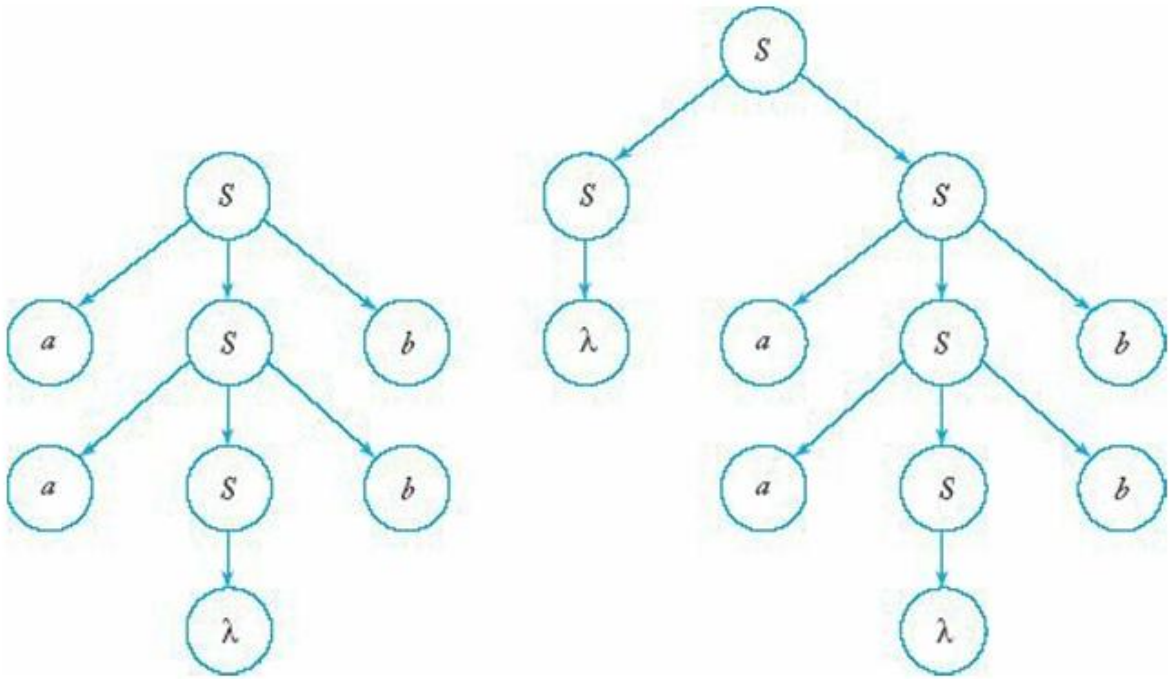
Ambigüedad en gramáticas

Una gramática formal es **ambigua** cuando existe al menos una cadena en su lenguaje que puede ser generada por **dos o más árboles de derivación (parse trees) distintos**. De forma equivalente, una gramática es ambigua si una cadena se puede derivar con **dos o más derivaciones más a la izquierda** (o más a la derecha) distintas.

Derivación más a la izquierda: Es aquella en la que, en cada paso, se reemplaza siempre la variable más a la izquierda.

Ambigüedad en gramáticas

La gramática con producciones S → aSb|SS|λ, es ambigua. La cadena aabb tiene estos dos árboles de derivación



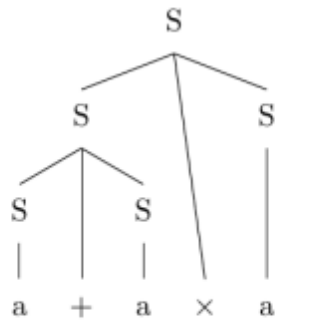
Ambigüedad en gramáticas

Consideremos la gramática con reglas:

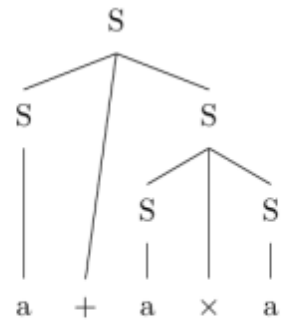
$$E \rightarrow E + E \mid E * E \mid a$$

La cadena $a + a * a$ puede *interpretarse* de dos maneras:
$(a + a) * a$ o $a + (a * a)$

Cadena	Derivación
$a + a * a$	$S \Rightarrow S + S \Rightarrow a + S \Rightarrow a + S * S \Rightarrow a + a * S \Rightarrow a + a * a$
$a + a * a$	$S \Rightarrow S * S \Rightarrow S + S * S \Rightarrow a + S * S \Rightarrow a + a * S \Rightarrow a + a * a$



Árbol 1: Suma primero
Interpretación: $(a + a) \times a$



Árbol 2: Multiplicación primero
Interpretación: $a + (a \times a)$

Ambigüedad en gramáticas

Los lenguajes de programación necesitan gramáticas sin ambigüedad. Porque de otro modo, distintos compiladores pueden hacer derivaciones distintas para la misma instrucción, obteniendo interpretaciones (semántica) distintas.

Hay lenguajes que son inherentemente ambigüos (no existe una gramática no ambigüa que los genere)

Ejemplo: $L = L_1 \cup L_2$ es un LIC inherentemente ambigüo,
donde $L_1 = \{a^n b^n c^m, n, m \geq 0\}$ y $L_2 = \{a^n b^m c^m, n, m \geq 0\}$

Ambigüedad en gramáticas

Para eliminar ambigüedad en la gramática con variable inicial E:

$$E \rightarrow E + E \mid E * E \mid a$$

se puede asociar niveles de precedencia al nivel de variables.

$$E \rightarrow E + T \mid \mathrm{T}$$
$$T \rightarrow T * \mathrm{F} \mid \mathrm{F}$$
$$F \rightarrow a \mid (\mathrm{E})$$

En esta gramática, cada operador vive en un nivel sintáctico diferente:

- La suma $(+)$ se procesa en el nivel superior (E).
- La multiplicación $(*)$ se procesa en un nivel inferior (T).

Así: una multiplicación no compite al mismo nivel que una suma.

Para que una expresión contenga ambos operadores, **la suma está obligada a ser el nodo superior**, mientras que la multiplicación queda atrapada y subordinada dentro del término (T).

Problemas de decisión

Un *problema de decisión* para cierta propiedad *P* de gramáticas, consiste en buscar un algoritmo, aplicable a una GIC arbitraria G, que responda SI o NO a la pregunta: ¿satisface G la propiedad P?

Un problema es decidible si existe tal algoritmo que permita responder la pregunta.

Sí... hay problemas no decidibles! Preguntas que no se pueden contestar en el caso general...

El problema de decidir: $w \in L(G)$

Dada una cadena w de terminales, ¿cómo determinar si puede ser generada por una gramática G?.

Ejemplo: Consideremos la gramática libre de contexto
$G = (V, \Sigma, R, S)$, donde:
Conjunto de variables (V): $\{S\}$
Alfabeto de terminales (Σ): $\{a, b, c\}$
Símbolo inicial (S): S
Reglas de producción (R):

$$S \to aSbS$$
$$S \to c$$

La cadena w = acbaacbbc se puede generar con G?

El problema de decidir: $w \in L(G)$

Dada una cadena w el análisis sintáctico exhaustico consiste en:
Generar todas las cadenas en un paso. ¿Alguna es w? Si no, continuar
Generar todas las cadenas en dos pasos. ¿Alguna es w? Si no, continuar
... etc...

Si $w \in L(G)$, el proceso termina. Es un proceso tedioso, pero termina.
Si $w \notin L(G)$ … puede no terminar.

Teorema: Si una GIC G no tiene reglas de la forma:

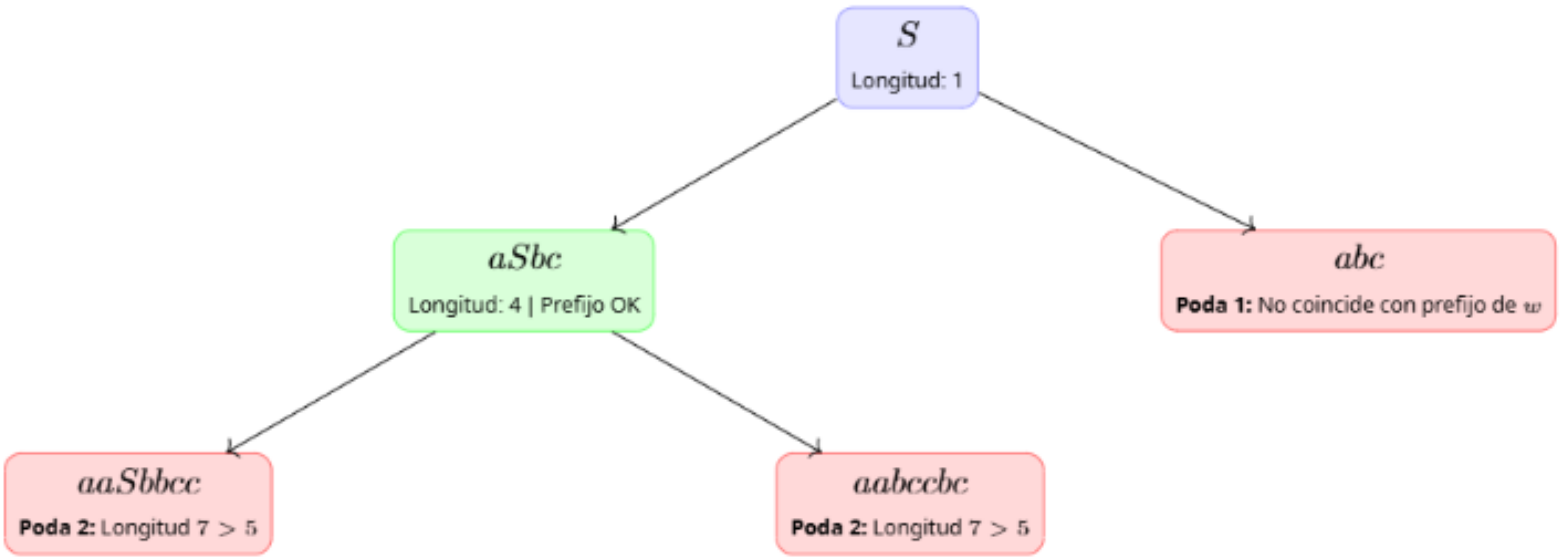
$$A \to \lambda$$
$$A \to B$$

el proceso de análisis sintáctico exhaustivo de una cadena w siempre termina decidiendo la pertenencia o no a L(G) (en cantidad finita de pasos)

El problema de decidir: $w \in L(G)$

Ejemplo: Realizamos el análisis sintáctico exhaustivo para la cadena *w* = *aabcc* con la gramática dada por las reglas

$$S \to aSbc$$
$$S \to abc$$



El problema de decidir: $w \in L(G)$

Ejemplo: Realizamos el análisis sintáctico exhaustivo para la cadena *w* = *ababcc* con la gramática dada por las reglas

$$S \to XSc$$
$$S \to abc$$
$$X \to aXb$$
$$X \to ab$$

El problema de decidir: $w \in L(G)$

Ejemplo: Realizamos el análisis sintáctico exhaustivo para la cadena *w* = *abacc* con la gramática dada por las reglas

$$S \to XSc$$
$$S \to abc$$
$$X \to aXb$$
$$X \to ab$$

El problema de decidir: $w \in L(G)$

El algoritmo de decisión para el problema P: dada G y w, $w \in L(G)$? (análisis sintáctico exhaustivo) tiene complejidad exponencial.

Existe otro algoritmo (CYK) de *complejidad polinomial* basado en tabulación dinámica, pero no lo veremos aquí.

Problemas decidibles con GR

Dada una gramática regular G, cómo se pueden responder las cuestiones:

1. ¿Es L(G) no vacío? ¿Es infinito?

Para responder: se construye el AF M equivalente. Si existe un camino de q_0 a un estado final, entonces L(M), y por lo tanto, L(G) es no es vacío.

Cuándo L(M) es infinito?

Problemas decidibles con GR

Dada una gramática regular G, cómo se pueden responder las cuestiones:

1. ¿Es L(G) no vacío? ¿Es infinito?

Para responder: se construye el AF M equivalente. Si existe un camino de q_0 a un estado final, entonces L(M), y por lo tanto, L(G) es no es vacío.

Cuándo L(M) es infinito?

Si y sólo si existe algún ciclo en el grafo correspondiente a M.

Problemas decidibles con GR

2. Dadas dos gramáticas regulares G_1 y G_2, $L(G_1)=L_1$ y $L(G_2)=L_2$ son iguales?

Para responder: se construye el AF M para el lenguaje

$$L_3 = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$$

Problemas decidibles con GR

2. Dadas dos gramáticas regulares G_1 y G_2, $L(G_1)=L_1$ y $L(G_2)=L_2$ son iguales?

Para responder: se construye el AF M para el lenguaje

$$L_3 = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$$

Si $L_3 = \varnothing$, entonces $L_1 = L_2$

Problemas indecidibles con GIC

Hay problemas referentes a gramáticas de tipo 2, 1 o 0 que son indecidibles, es decir, problemas para los cuales no existen algoritmos de decisión.

Los siguientes problemas son indecidibles:

1. Dada una gramática G, ¿es G ambigua?
2. Dada una gramática G, ¿genera G todas las cadenas de terminales?, es decir, ¿$L(G)=\Sigma^*$?
3. Dadas dos gramáticas G_1 y G_2, ¿generan G_1 y G_2 el mismo lenguaje?, es decir, ¿$L(G_1)=L(G_2)$?

Bibliografía



Rodrigo De Castro Korgi.

“Teoria de la computación”. Lenguajes, autómatas, gramáticas.



Peter Linz.

“An introduction to formal languajes and automata”.

Capítulo 5.